

Reel Notes — Index

2 reel(s)

1. p.2 How can the cost of running large language models be reduced as user demand scales from 1,000 to 100,000 users?

2. p.6 How does a decision tree use Gini impurity to select the root node and build subsequent splits to predict whether a student will pass or fail an AI certification?

Question

How can the cost of running large language models be reduced as user demand scales from 1,000 to 100,000 users?

Video by codedsoul_05

[https://www.instagram.com/reel/DXhDf3JEzCi/?](https://www.instagram.com/reel/DXhDf3JEzCi/?utm_source=ig_web_copy_link&igsh=MzRIODBiNWFIZA==)

[utm_source=ig_web_copy_link&igsh=MzRIODBiNWFIZA==](https://www.instagram.com/reel/DXhDf3JEzCi/?utm_source=ig_web_copy_link&igsh=MzRIODBiNWFIZA==)

Video by codedsoul_05

1. Overview

The reel discusses the factors contributing to the increasing cost of large language models (LLMs) as the number of users grows. It highlights that the primary driver of cost is not the number of users, but rather the number of tokens processed. The video presents a mini-framework for reducing costs, focusing on techniques such as trimming system prompts, dropping redundant examples, and capping maximum output tokens. By applying these strategies, teams can achieve significant cost reductions, ranging from 40-60% immediately.

2. Key Concepts

- **Token Explosion Effect:** The phenomenon where 80-95% of the bill is attributed to input tokens, with teams often sending 2,500-token prompts for 300-token answers.
- **The Context Window Tax:** The idea that every extra word is rented compute, resulting in significant costs, especially when dealing with a large number of users.
- **Tokenization Mismatch:** The difference in tokenization across models, such as tiktoken-style encoders used in OpenAI models and Toon tokenizer used in Anthropic models.
- **Prompt Optimization Lever:** The use of tools, such as Anthropic's prompt optimization tools, to compress verbose instructions, reduce hallucinations, and standardize outputs.

3. Detailed Explanation

3.1 The Token Explosion Effect

The Token Explosion Effect occurs when the number of input tokens grows exponentially with the number of users. This is because teams often send lengthy prompts, which can be up to 2,500 tokens, to generate answers that are only 300 tokens long. To mitigate this, teams can trim system prompts, drop redundant examples, and cap maximum output tokens. By doing so, they can reduce the number of input tokens, resulting in significant cost savings.

3.2 The Context Window Tax

The Context Window Tax refers to the cost associated with processing every extra word in a prompt. This cost can be substantial, especially when dealing with a large number of users. For instance, if a team has 100,000 users and each user generates 4,000 tokens per day, the total number of tokens processed per day would be 400 million. At a cost of 8 – 15 per 1 million tokens, this would translate to a daily cost of 3,000 – 6,000, or approximately 90,000 – 180,000 per month.

3.3 Tokenization Mismatch

Tokenization mismatch occurs when different models use different tokenization techniques, resulting in varying token counts for the same prompt. For example, OpenAI models use tiktoken-style encoders, while Anthropic models use Toon tokenizer. This mismatch can lead to hidden cost variance per request, as the same prompt can be tokenized differently across models. To address this, teams can measure tokens using model-native tokenizers, rewrite prompts to minimize whitespace and formatting, and benchmark the same prompt across models to compare real token counts.

3.4 Prompt Optimization Lever

The Prompt Optimization Lever refers to the use of tools, such as Anthropic's prompt optimization tools, to optimize prompts and reduce the number of tokens required to generate answers. These tools can compress verbose instructions, reduce hallucinations, and standardize outputs, resulting in significant token savings. Teams that have used these tools have reported token savings of 20-40%, which can translate to substantial cost reductions, especially at scale.

4. Step-by-Step

To reduce costs, teams can follow these steps: 1. **Measure baseline**: Record the current number of tokens processed, GPU memory usage, and cost per 1,000 requests. 2. **Select a compression technique**: Choose a technique, such as trimming system prompts, dropping redundant examples, or capping maximum output tokens. 3. **Validate quality**: Test the compressed prompts on a representative prompt set to ensure accuracy drop is less than 2%. 4. **Implement KV caching**: Integrate a cache layer in the inference server to reuse context across requests. 5. **Enable dynamic batching**: Configure the serving stack to batch incoming requests automatically. 6. **Deploy horizontally**: Set up an auto-scaling group of inference containers behind a load balancer. 7. **Monitor cost metrics**: Track GPU hours, request latency, and per-request cost, and adjust instance count or batch size as needed.

5. Practical Takeaways

- **Cost grows linearly with token count and model size**: Reducing either directly lowers expense.
- **Model-level tricks**: Distillation, quantization, pruning, and MoE can shrink compute required per token.
- **Inference-level tricks**: Batching, KV caching, and prompt compression can reduce active computations per request.
- **System-level scaling**: Horizontal and vertical scaling, as well as efficient hardware usage, can ensure the service can handle higher demand without proportional cost spikes.
- **Measure, validate, and monitor**: Always track key metrics to avoid sacrificing quality or user experience while cutting costs.

6. Further Notes

When optimizing LLM costs, it is essential to consider the trade-offs between quality, latency, and throughput. Aggressive quantization or pruning can degrade output fidelity, while batching can improve throughput but increase per-request latency. Teams should validate the impact of any optimization technique on their specific use case and adjust their approach accordingly. Additionally, exploring different tokenization techniques and optimizing prompts can lead to significant cost savings. By applying these strategies and considering the unique characteristics of their LLM deployment, teams can reduce costs while maintaining acceptable quality and user experience.

Question

How does a decision tree use Gini impurity to select the root node and build subsequent splits to predict whether a student will pass or fail an AI certification?

Video by aicoreknowledge

[https://www.instagram.com/reel/DasfzZBxRTg/?](https://www.instagram.com/reel/DasfzZBxRTg/?utm_source=ig_web_copy_link&igsh=MzRIODBiNWFIZA==)

[utm_source=ig_web_copy_link&igsh=MzRIODBiNWFIZA==](https://www.instagram.com/reel/DasfzZBxRTg/?utm_source=ig_web_copy_link&igsh=MzRIODBiNWFIZA==)

Video by aicoreknowledge

1. Overview

The reel by aicoreknowledge focuses on explaining the Decision Tree Classifier in Machine Learning, specifically how to derive the root node and create a decision tree using a sample dataset. The goal is to predict whether a student will pass or fail an AI certification based on features like practice, study hours, and attendance. By the end of this explanation, the reader will understand how to apply the Gini impurity measure to select the best attribute for the root node and recursively build the decision tree.

2. Key Concepts

- **Decision Tree Classifier:** A machine learning model that uses a tree-like structure to classify data.
- **Gini Impurity:** A measure of how well a dataset is separated into classes, with lower values indicating better separation.
- **Root Node:** The topmost node in the decision tree, which represents the best attribute to split the data.
- **Weighted Gini:** A calculation used to determine the Gini impurity of an attribute, taking into account the proportion of samples in each child node.
- **Pure Node:** A node where all samples belong to the same class, resulting in a Gini impurity of 0.

3. Detailed Explanation

Introduction to Decision Trees

Decision trees are a type of supervised learning algorithm that can be used for both classification and regression tasks. They work by recursively partitioning the data into smaller subsets based on the values of the input features. The decision tree classifier is particularly useful for handling categorical data and for providing interpretable results.

Understanding Gini Impurity

Gini impurity is a measure of how well a dataset is separated into classes. It is calculated using the formula:

$$Gini = 1 - \sum (p_k^2)$$

where p_k is the proportion of samples in the k^{th} class. The Gini impurity ranges from 0 (perfectly pure) to 0.5 (maximum mixing), with lower values indicating better separation.

Deriving the Root Node

To derive the root node, we need to calculate the weighted Gini impurity for each attribute. The attribute with the lowest weighted Gini impurity is chosen as the root node. The weighted Gini impurity is calculated using the formula:

$$Gini_{feature} = \sum_v \frac{N_v}{N} \times Gini(v)$$

where N_v is the number of samples in the v^{th} child node, N is the total number of samples in the parent node, and $Gini(v)$ is the Gini impurity of the v^{th} child node.

Example Calculation

Using the sample dataset provided, let's calculate the weighted Gini impurity for the "Practice" attribute. The dataset contains 12 students, with 5 students who did not practice and 7 students who did practice. The Gini impurity for the "No" branch is 0, since all students who did not practice failed. The Gini impurity for the "Yes" branch is:

$$Gini = 1 - \left(\frac{5}{7}\right)^2 - \left(\frac{2}{7}\right)^2 = 0.408$$

The weighted Gini impurity for the "Practice" attribute is:

$$Gini_{Practice} = \frac{5}{12} \times 0 + \frac{7}{12} \times 0.408 = 0.238$$

Selecting the Root Node

The attribute with the lowest weighted Gini impurity is chosen as the root node. In this case, the "Practice" attribute has the lowest weighted Gini impurity, so it is chosen as the root node.

4. Step-by-Step

1. Prepare the dataset by selecting the relevant features and target variable.
2. Calculate the Gini impurity for each attribute using the formula: $Gini = 1 - \sum (p_k^2)$.
3. Calculate the weighted Gini impurity for each attribute using the formula: $Gini_{feature} = \sum_v \frac{N_v}{N} \times Gini(v)$.
4. Select the attribute with the lowest weighted Gini impurity as the root node.
5. Recursively apply the same process to each child node until all nodes are pure or no further splits improve purity.
6. Use the resulting decision tree to make predictions by traversing from the root node to a leaf node based on the feature values of the input instance.

5. Practical Takeaways

- Use the Gini impurity measure to select the best attribute for the root node.
- Recursively apply the same process to each child node to build the decision tree.
- Use the resulting decision tree to make predictions by traversing from the root node to a leaf node.
- Be aware of the potential for overfitting and take steps to prevent it, such as pruning the tree